

Phase 1: Requirements & Workflow Analysis for NovaCRM

1. Current State Workflow & Bottleneck Analysis

The following table breaks down the current operational flow, the specific failure modes identified by the CS Manager (Priya), and the architectural solutions required.

Current Workflow Step	Operational Bottleneck & Failure Mode	Architectural Solution
Deal Notification	AE sends an unstructured email containing only the customer name and Salesforce link, explicitly omitting the plan tier (Enterprise vs Growth).	Agent 1 (Intake Engine): Enforces strict schema extraction via webhook listener to capture the customer name, AE name, and the explicitly required AE phone number.
Project Provisioning	CSMs manually create Asana projects, frequently guessing the plan tier and assigning the wrong template (mixing up 14-day pooled vs 30-day dedicated frameworks).	Agent 1 (Voice AI Trigger): Pauses provisioning to call the AE's phone to verbally confirm the tier. Once confirmed, executes a deterministic Rocketlane API call using the exact template ID.
Communication Setup	Manual creation of a shared Slack channel and ad-hoc kickoff scheduling.	Agent 2 (Communication Agent): Listens for the Rocketlane project creation webhook to programmatically provision and personalize the Slack channel.
Task Escalation	SLA breaches (tasks blocked > 3 days) rely on manual monitoring and frequently fall through the cracks.	Rocketlane Native Automation: In-app rules automatically trigger alerts at 1 day (Project Manager) and 4 days (Project Owner)

		overdue.
Data Migration	Tasks are marked "done" without verification, causing downstream escalations due to corrupt or missing data.	Data Migration Gatekeeper (Agent 3): Hard phase-gate dependency requiring explicit verification sign-off before unlocking Phase 3 (Configuration).

2. Proposed Agentic Workflow Architecture

The target state implements a continuous, event-driven multi-agent system utilizing intelligent modules and platform-native logic to satisfy the assignment criteria.

A. Intake & Ingestion Engine (Agent 1 - Part A)

Agent 1 monitors the inbound CS email queue. Upon receiving a notification, it parses the payload against a strict JSON schema requiring customer_name, customer_contact_email, ae_name, and the hard-required ae_phone_number.

- **Guardrail:** If the required fields are present, the agent stages the payload. If malformed or missing data, it drafts an immediate clarification reply to the AE rather than guessing.

B. Telephony Tier Confirmation (Agent 1 - Part B)

Because the inbound email omits the subscription plan, Agent 1 cannot select the onboarding template. It initiates an outbound webhook to a Voice AI provider, passing the AE's extracted phone number to verbally confirm if the customer is on the Enterprise or Growth plan.

- **Guardrail (Escalation Fallback):** If the AE fails to answer, drops the call, gives an ambiguous response, or the system hits its retry limit, the agent does not guess. It immediately halts the workflow, generates a high-priority ticket/alarm to the human AE indicating manual tier confirmation is required.

C. Rocketlane Workspace Provisioning

Upon receiving positive, unambiguous verbal confirmation of the tier, Agent 1 structures an API payload to the Rocketlane API.

- **Routing Logic:**
 - If **Enterprise**: Injects the Template ID for the 30-day framework and provisions a dedicated CSM assignment.
 - If **Growth**: Injects the Template ID for the 14-day framework and assigns it to the

pooled CS queue.

D. Collaboration Provisioning (Agent 2)

Agent 2 operates as the Communication Agent. It securely interfaces with the Slack API to provision a shared channel for onboarding communication. The channel name, topic, and initial welcome message are formatted dynamically based on the plan tier and customer details.

E. Overdue Task Escalation (Rocketlane Native)

Instead of building a separate AI agent, we configure the platform's native automation engine to handle escalations.

- **1-Day Overdue:** Rule triggers a notification to the Project Manager.
- **4-Days Overdue:** Rule triggers a notification to the Project Owner.

3. Strict Technical & Guardrail Requirements

To successfully deliver the architecture outlined in the assignment, the following system guardrails must be explicitly coded into the solution.

- **Idempotency & Duplicate Prevention:** The system must handle the edge case where a customer already has an existing onboarding project.
- **System Outage Handling:** The architecture must gracefully handle scenarios where the Rocketlane API is completely down.
- **Mandatory Comprehensive Logging:** Every single agent action must be logged, specifically including timestamps, inputs, outputs, and the decision rationale.
- **Live Telephony Requirement:** The Voice AI implementation must be a real, working voice interaction over the phone. Text simulations are explicitly forbidden.
- **Video Deliverables Constraints:** The final submission must be a screen-recorded video showcasing two specific runs: a complete "Positive sample" (happy path) and a "Negative sample" demonstrating failures, retries, and guardrails (e.g., malformed email, AE doesn't answer).
- **Mocking Permissibility:** Using a simulated/mock API for Slack is acceptable, provided you verbally explain in the video what would change in a real production deployment.

4. Clarifying Questions

The following operational and technical assumptions require clarification to accurately finalize the deliverables for the assignment.

1. Slack Workspace Boundaries (Agent 2)

- **Clarification Needed:** For the Communication Agent's requirement to create a "shared Slack channel with the customer", does this strictly require executing a cross-domain Slack Connect invitation to an external organization, or is it acceptable to simulate the customer by inviting an internal dummy email user within a single sandbox workspace?

2. Rocketlane Template Prerequisites

- **Clarification Needed:** In the free Rocketlane trial account, are we expected to manually pre-build the 14-day (Growth) and 30-day (Enterprise) templates in the UI and simply reference their static Template IDs in the API call, or must the Intake Agent programmatically generate the 15 tasks and 4 phases via the API dynamically for every project?

3. Inbound Email Payload Discrepancy

- **Clarification Needed:** The discovery notes state the AE's email only contains "the customer name and the Salesforce opportunity link". However, Part 2 states the agent must parse "the customer name, customer contact email, AE name, and other details". Should we assume for this assignment that our test emails will explicitly include the contact email and AE name in the text body, or is the agent expected to simulate scraping the Salesforce opportunity link to find those missing fields?

4. Automated Tests vs. Video Demonstration Deliverables

- **Clarification Needed:** Part 4 explicitly asks to "Write automated tests" for various paths and edge cases, but the Deliverables section only requests a "video walkthrough demonstrating two scenarios" (one positive, one negative). Do you require actual test scripts (e.g., Pytest/Jest files) submitted as a code payload, or is verbally walking through the test scenarios manually in the Loom video sufficient?

5. Audit Logging Destination

- **Clarification Needed:** Part 3 requires that "All agent actions should be logged with timestamps, inputs, outputs, and decision rationale". For the scope of this take-home, is it sufficient to output these structured logs to the local terminal/console, or must they be written to a persistent external database/file (like a Google Sheet or SQLite db) to be shown in the video?

6. Framework Constraints (LangChain / Framework Usage)

- **Clarification Needed:** Is it strictly required to use LangChain or an equivalent orchestration framework, or can this assignment be completed without using one (e.g., using native API calls and standard Python libraries)?

7. Voice AI API Costs & Credit Requirements

- **Clarification Needed:** No voice AI agent performs free outbound calls, and developers are typically required to purchase platform credits (e.g., Vapi, Bland AI, or Retell). Is it normal to be expected to purchase credits to fulfill this take-home assignment?

5. Documented System Assumptions

In designing this multi-agent architecture, the following technical and operational assumptions were made to proceed with the build:

1. AE Phone Number Availability (Hard Requirement)

- **Assumption:** We assume the incoming deal notification email reliably contains the AE's phone number, or that the system has access to a deterministic internal directory (e.g., a lightweight database or JSON file) to map the parsed ae_name to their direct phone line.

This is a hard requirement for the Voice AI outbound trigger.

2. Escalation & Fallback Protocol

- **Assumption:** We assume that if the Voice AI system reaches its maximum retry limit (e.g., 3 failed calls) or the AE provides a persistently ambiguous response, the system should **not** default to a fallback template. Instead, it assumes a "fail-safe" posture: halting automation and automatically initiating a support ticket/alarm alerting the human AE that manual intervention is required.

3. Idempotency Key Generation

- **Assumption:** We assume that a combination of the parsed customer_name and the incoming email's Message-ID (or Salesforce Opportunity Link) will serve as a reliable, unique primary key. This key is necessary to ensure idempotency, preventing the system from creating duplicate Rocketlane projects if an AE accidentally forwards the deal email twice.

4. Voice AI Asynchronous Architecture

- **Assumption:** We assume the chosen Voice AI provider (e.g., Vapi, Bland AI, or Retell) supports asynchronous webhook callbacks. Agent 1 will trigger the call via a REST API, and the Voice AI will autonomously manage the telephony state, returning a structured JSON payload (transcript, call status, and the extracted tier value) to our server once the call terminates.

5. Slack API Permissions

- **Assumption:** We assume the Slack Bot Token provisioned for Agent 2 has the scopes necessary to execute conversations.create, conversations.invite, and chat.postMessage.

6. Logging

- **Assumption:** Currently logging is a function that'll be called whenever the agent logging is required. It can be repurposed into an event where all the loggings are saved into a database.